

Python para Engenharia de Dados

Consumo de APIs REST em Escala

Conceito Detalhado

No ecossistema de dados moderno, grande parte da ingestão das camadas iniciais (Bronze/Raw) provém do consumo de **APIs REST** de sistemas operacionais, gateways de pagamento e CRMs.

- **Verbs HTTP em Ingestão:** O foco principal da engenharia de dados é o método GET (extração de dados), podendo utilizar POST para autenticações de sessão.
- **Autenticação:** As APIs corporativas protegem seus endpoints através de tokens de longa/curta duração no cabeçalho (*Header*):
 - *Bearer Token:* Authorization: Bearer <seu_token>
 - *OAuth2:* Troca de credenciais do cliente (client_id e client_secret) por um *access token* com tempo de expiração.
- **Paginação:** Como as origens não retornam milhões de registros em uma única resposta, os dados são entregues em páginas. O pipeline precisa iterar utilizando estratégias baseadas em:
 - *Offset / Limit:* Ex: ?limit=100&offset=200
 - *Page Number:* Ex: ?page=3
 - *Cursor / Next URL:* A API fornece um ponteiro ou link direto para a próxima página no JSON da resposta.
- **Rate Limiting e Resiliência:** Servidores limitam o número de requisições por minuto (Status HTTP 429 Too Many Requests). O pipeline de dados deve tratar o erro aplicando a técnica de **Exponential Backoff** (aguardar tempos crescentes como 2s, 4s, 8s antes de tentar novamente).

Manipulação e Conversão de Arquivos (CSV, JSON, Parquet, Avro)

Comparativo de Formatos no Data Lake

Formato	Orientação	Compressão / Performance	Tipo de Arquivo	Melhor Uso em Data Engineering
CSV	Linha	Ruim (Texto sem Aberto, tipagem)	Humano / Texto	Troca simples de arquivos em legado
JSON	Hierárquico	Médio (Chaves repetidas)	Humano / Texto	Ingestão Bruta (Camada Bronze) / APIs

Parquet	Coluna	Excelente (Snappy/Gzip, <i>Pushdown</i>)	Binário	Camadas Silver e Gold / OLAP
Avro	Linha	Alta (Baseado em Schema JSON)	Binário	Streaming / Kafka / Ingestão Bronze

Engenharia de Software Aplicada ao Python

- **Evitando o print():** O uso de print() grava mensagens diretamente no stdout sem nível de severidade, carimbo de data/hora ou estruturação. Em produção, utiliza-se a biblioteca logging configurada para emitir logs com contexto (DEBUG, INFO, WARNING, ERROR, CRITICAL) em formato JSON estruturado.
- **Gestão de Erros (try/except/raise):** Capturar exceções específicas (ex: requests.exceptions.HTTPError) e forçar a falha explícita (raise) quando um contrato crítico for quebrado.

Pipeline de Dados (11 horas)

Ciclo de Vida do Dado: ETL vs. ELT

- **ETL (Extract, Transform, Load):**
 - *Fluxo:* O dado é extraído da origem, transformado no servidor/engine de processamento (ex: Python/Spark) e **depois** gravado limpo no destino final.
 - *Uso:* Data Warehouses legados com limitação de processamento interno ou quando dados sensíveis (LGPD/CPF) precisam ser mascara/anônimizados antes de tocar o destino.
- **ELT (Extract, Load, Transform):**
 - *Fluxo:* O dado é extraído e gravado **diretamente no seu estado bruto** dentro do Data Lakehouse (Camada Bronze). A transformação ocorre **dentro do próprio destino** aproveitando seu poder computacional distribuído.
 - *Uso:* Padrão da arquitetura moderna de dados (Lakehouse, Snowflake, Databricks, BigQuery).

Idempotência, Backfill e Reexecução

- **Princípio da Idempotência:** Uma operação é idempotente se a sua execução **múltiplas vezes produz exatamente o mesmo resultado** de uma única execução.
 - *Errado:* Executar o script diariamente fazendo INSERT INTO (gera duplicatas a cada tentativa de correção).

- *Correto*: Executar substituição de partições (OVERWRITE PARTITION) ou estratégias de mesclagem (MERGE / UPSERT baseadas na Chave Primária).
- **Backfill (Reprocessamento Histórico)**: A capacidade de orquestrar a reexecução do pipeline para intervalos de datas passados (ex: recalculando todo o mês de agosto devido a uma alteração na regra de negócio) sem interromper a carga atual.

Orquestração e DAGs (Directed Acyclic Graphs)

As tarefas de um pipeline não rodam isoladas. Elas formam um **Grafo Acíclico Dirigido (DAG)**, onde a Tarefa B depende do sucesso da Tarefa A, sem criar loops infinitos.